# Vizora

# Dataset Preparation and Data Handling for Model Development

A project document for the Gridlock Hackathon submission covering data sourcing, normalization, quality control, privacy safeguards, and training-ready dataset assembly.

Prepared by: Mohana Krishna

Contact: codexmohan@gmail.com

Flipkart Gridlock 2.0 Hackathon - Round 2

| Focus Area | Project Position |
| --- | --- |
| Primary goal | Create a reliable, privacy-conscious data pipeline for traffic violation detection from still images and near-real-time camera feeds. |
| Training formats | Canonical YOLO datasets for detection, optional COCO exports for interoperability, plus targeted plate and seatbelt subsets. |
| Repository alignment | Matches the training pipeline in training/scripts/prepare_datasets.py and the model stack described in PLAN.md and README.md. |
| Core principle | Data handling must support both image-first hackathon evaluation and deployment-style live surveillance extensions. |

## 1. Scope and Objectives

Vizora is built around automated photo identification and classification for traffic violations, which makes dataset preparation a first-class system component rather than a side task. The project needs data that supports object detection, violation reasoning, plate extraction, and review-safe evidence generation under real road conditions such as crowding, occlusion, compression artifacts, perspective distortion, haze, rain, and low-light CCTV footage.

- Prepare reusable training sets for traffic participant detection, license plate localization, helmet or headwear cues, and seatbelt evidence.
- Normalize mixed public and custom sources into a consistent class taxonomy that the training code and inference pipeline can trust.
- Filter weak labels and invalid boxes early so downstream model behavior is not dominated by annotation noise.
- Preserve privacy by treating raw evidence, plates, and reviewer copies differently from public or demo outputs.

## 2. Dataset Strategy

The repo already anticipates a mixed-source dataset strategy. Public detection sources are useful for vehicle diversity and base initialization, while custom data is required for traffic-rule semantics, Indian road scenes, regional plates, and edge cases such as turbans, child-on-lap situations, emergency vehicles, and stop-line context. Vizora therefore treats data preparation as a staged funnel: acquire, map, clean, split, validate, and export.

| Dataset track | Purpose | Expected source pattern | Target output |
|---|---|---|---|
| Traffic objects | Vehicles, riders, pedestrians, bicycles, plates-in-scene | Public COCO-style or YOLO traffic datasets plus curated local captures | Canonical YOLO plus optional COCO export |
| Plate dataset | License plate localization and OCR crop generation | Indian plate imagery, public ANPR datasets, custom traffic frames | Plate-only YOLO dataset and crop manifests |
| Helmet or headwear | Helmet compliance and exemptions | Headwear datasets and custom two-wheeler captures | Merged YOLO labels aligned to canonical classes |
| Seatbelt evidence | Seatbelt presence in cars and cabs | Curated front-seat or side-angle images with visible occupants | Seatbelt-specific detection subset |

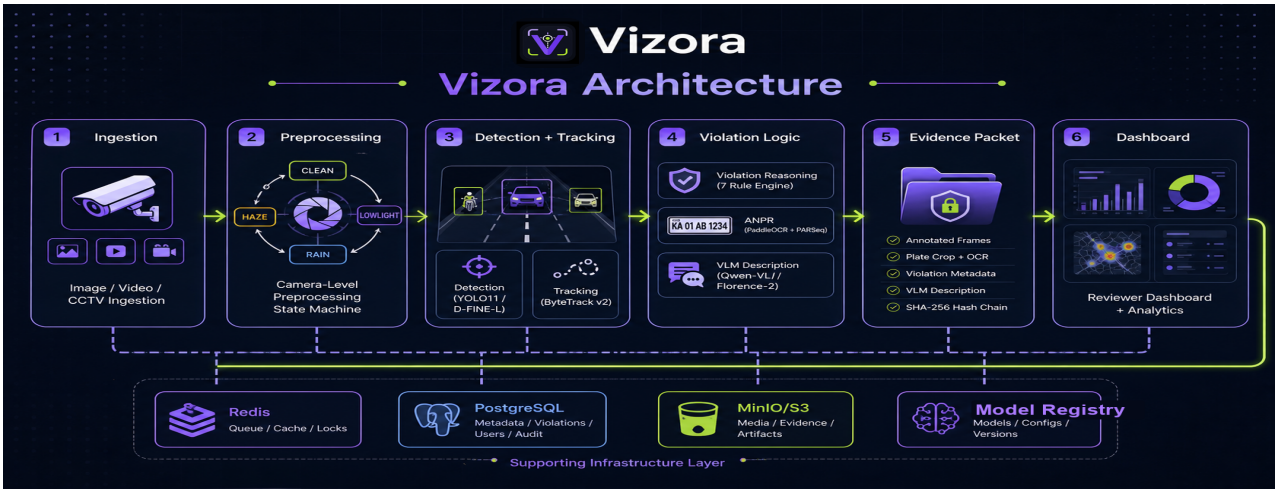

*Figure: Vizora system architecture, showing how training data preparation connects to inference, evidence, and review flows.*

## 3. Canonical Class Mapping

The preparation script normalizes heterogeneous source labels into a stable class vocabulary before training. This matters because real traffic datasets often use inconsistent names such as sedan, hatchback, MUV, mini-bus, LCV, or three-wheeler. Vizora maps them into canonical groups such as car, bus, truck, auto, motorcycle, bicycle, and license_plate so that models and evaluation reports remain comparable across sources.

- Canonical naming reduces class-fragmentation and improves sample counts per class.
- Label mapping is applied before writing YOLO labels, not after training, so every downstream artifact inherits the same taxonomy.
- Unmapped categories are counted and skipped explicitly, which is safer than silently forcing them into an incorrect class.

## 4. Preparation Workflow in the Repository

The file training/scripts/prepare_datasets.py already captures the backbone of the preparation workflow. It can synthesize small detection sets for fast testing, convert COCO annotations into canonical YOLO format, copy images into train and validation splits, filter invalid or tiny boxes, validate normalized labels, and emit conversion manifests for auditability. That script gives the project a real operational path for judge demos and future training iterations.

| Workflow stage | What happens | Why it matters |
|---|---|---|
| Ingest annotations | Read COCO JSON or source YOLO folders | Lets the project reuse existing traffic datasets instead of starting from scratch |
| Normalize labels | Map raw category names to Vizora classes | Keeps training and inference class ids aligned |
| Quality filtering | Drop invalid boxes, tiny boxes, and extreme aspect ratios | Prevents noisy or impossible labels from destabilizing training |
| Split export | Write train and validation images and labels | Creates deterministic dataset layouts for model training |
| Manifest and validation | Store conversion metadata and verify label integrity | Makes the preparation step reproducible and reviewable |

## 5. Quality Control and Filtering

Traffic data gets messy quickly: partial crops, compressed frames, inaccurate boxes, and mislabeled classes are common. Vizora's preparation layer should reject low-value annotations early so the model spends capacity on meaningful visual patterns. The current script already includes safeguards such as minimum box side, minimum area, maximum aspect ratio, and explicit counters for skipped annotations.

- Drop boxes that collapse after clipping, which usually indicates broken source geometry.
- Reject extremely tiny boxes because they rarely contain enough signal for reliable learning on compressed CCTV footage.
- Track skipped annotations in manifests so data loss is observable instead of hidden.
- Keep empty images only when they are intentionally useful as hard negatives; otherwise drop them to avoid confusing detector balance.

## 6. Split Policy and Leakage Prevention

A strong benchmark needs more than random file splitting. For surveillance imagery, near-duplicate frames from the same camera burst can leak across splits and inflate metrics. Vizora should therefore separate data by clip, burst, or camera-time window whenever possible, especially for live footage and downloaded traffic videos. The synthetic path in the repo uses a simple train and validation split, but the production training policy should be stricter.

| Rule | Recommended handling |
| --- | --- |
| Frame bursts | Keep consecutive frames from the same short sequence in a single split. |
| Camera views | Reserve at least one camera or intersection for holdout evaluation when enough data exists. |
| Plate crops | Do not split crops from the same original frame across train and validation. |
| Manual review cases | Maintain a dedicated difficult-set folder for edge cases and qualitative regression checks. |

## 7. Annotation Policy for Violations

Not every output in Vizora should be trained from the same annotation primitive. Object detection needs boxes, plate OCR needs localized crops and text labels, while temporal violations need track- or scene-level supervision. The dataset plan should therefore separate three label families: object labels, attribute labels, and event labels.

- Object labels: vehicles, riders, pedestrians, bicycles, buses, trucks, autos, and visible plates.
- Attribute labels: helmet present, helmet absent, seatbelt present, seatbelt absent, uncertain or occluded.
- Event labels: red-light crossing, stop-line crossing, wrong-side travel, triple-riding, illegal parking, and review-only ambiguous cases.

This separation avoids forcing a single detector to learn tasks that actually belong to different temporal or semantic levels. It also matches the overall Vizora architecture, where helmet and seatbelt cues can be handled differently from track-based violation reasoning.

## 8. Privacy, Security, and Evidence Handling

Data handling in this project is intentionally privacy-conscious. Raw surveillance frames may include faces, number plates, and location metadata, so the training and review workflow should preserve a distinction between internal model-development assets and externally shareable demonstration assets.

This is already consistent with the project's storage design in PostgreSQL and MinIO/S3-compatible storage and with the README emphasis on self-hosted inference.

- Store raw source images in restricted folders with clear provenance and capture-date metadata.
- Generate redacted copies for README, pitch-deck, and public demo usage whenever plates or faces are not essential.
- Hash manifests and keep conversion reports so prepared datasets can be traced back to their source batches.
- Avoid sending sensitive evidence to external API providers during the preparation and labeling loop.

## 9. Augmentation and Balancing

A traffic model for Indian roads must survive visual variability: glare, monsoon haze, motion blur, low bit-rate compression, clutter, and severe scale changes. Dataset augmentation should reflect those real deployment conditions instead of generic augmentation for its own sake. The rule is simple: augment toward observed camera failure modes, not away from them.

| Challenge | Useful augmentation or handling |
| --- | --- |
| Low-light and dull CCTV | Exposure, gamma, contrast, and blur variation aligned with the preprocessing stack |
| Rain and haze | Moderate weather corruption, not extreme synthetic artifacts |
| Small distant objects | Multi-scale training and careful retention of valid small boxes |
| Class imbalance | Targeted oversampling of rare classes or scenes rather than blind duplication |
| Plate readability | Crop-level enhancement and OCR-specific fallback data rather than full-frame over-processing |

## 10. Live Footage and Stream-Derived Data

Vizora also supports camera sources and near-real-time ingestion, which changes the way training data should be harvested. Stream-derived data is valuable for capturing authentic CCTV conditions, but it must be sampled deliberately. Pulling every frame is wasteful and creates redundancy. A better policy is to sample informative frames, flag uncertain cases, and promote review-worthy windows into labeling queues.

- Sample frames at controlled intervals from RTSP or recorded clips rather than dumping full streams into annotation tools.
- Prefer scene-diverse snapshots: different traffic density, weather, times of day, and signal phases.
- Save camera id, source timestamp, and approximate location because those fields are useful for split logic and hotspot evaluation later.
- Route low-confidence or missed cases from the live demo into a hard-example queue for later labeling and retraining.

## 11. Evaluation Data Handling

Evaluation should not be treated as just another split. Vizora needs both metric-friendly validation data and a high-signal review set for real-world behavior checks. Detection tasks can use mAP-style metrics, while plate OCR, helmet classification, and temporal violations should each maintain their own evaluation slices. The important point is that the evaluation set must reflect the ugly realities of traffic footage, not only the cleanest frames.

- Maintain per-task validation subsets: traffic detection, plate localization, OCR crops, helmet cues, seatbelt cues, and temporal violation windows.
- Track camera-level performance because a model that averages well can still fail on one critical scene type.
- Preserve a qualitative error bank of false positives, false negatives, and ambiguous cases for demo iteration and judge questions.
- Keep benchmark manifests versioned so model comparisons are reproducible across experiments.

## 12. Assumptions and Immediate Next Steps

This document assumes a mixed-source dataset built from public traffic corpora, public or custom plate imagery, and curated local traffic footage. It also assumes that not all edge cases will be fully labeled before the hackathon demo. That is acceptable as long as the preparation process is disciplined, documented, and expandable. Vizora already has a practical preparation backbone in the repo; the next step is to feed it with stronger custom data for Indian road rule nuances and to formalize the difficult-case review loop.

- Use the existing preparation script as the canonical entry point for dataset conversion and validation.
- Add custom capture batches for Indian intersections, two-wheeler density, and readable as well as unreadable plates.
- Create annotation guidelines for exemptions and ambiguous cases before scaling manual labeling.
- Version every prepared dataset export with manifests so the training lineage stays audit-friendly.

*End of document*